Oceny, wspólne funkcje i tabela wyników

Rozbudujesz kalkulator sprawdzianu z pakietu startowego lekcji 3. Zachowasz sprawdzanie danych, komunikat o zaliczeniu oraz czyszczenie pól. Nowe funkcjonalności dodasz do tego samego projektu, w kolejności opisanej w tej instrukcji.

Efekt końcowy
Dwa niezależne kalkulatory sprawdzianów, oceny 1-6, wspólna logika obliczeń oraz tabela wyników 0-100%, której oceny uzupełnia pętla for. HTML określa strukturę, Bootstrap wygląd, a JavaScript wykonuje obliczenia i obsługuje kliknięcia.

Punkt startowy

Otwórz folder **lesson3/start** i pracuj na jego kopii. Masz index.html, styles.css oraz script.js z działającą logiką lekcji 2, podłączoną do pól matematyki z prefiksem math-. HTML zawiera kalkulator matematyki, skalę ocen, przycisk „Trzy przykłady”, akapit examples i tabelę wyników z przyciskiem. Jedyną sekcją, którą dodasz w HTML, będzie kalkulator języka polskiego. Pozostałe zadania wykonasz w JS.

Praktyczne przypomnienie

1. Sprawdź wcześniejszą funkcjonalność.
40/50 ma dać 80% i zielony komunikat; 10/50 ma dać 20% i czerwony komunikat. Dla 60/50 pokaż błąd, a po kliknięciu „Wyczyść” oba pola mają być puste.

2. Dopisz niewielkie rozszerzenie.
Używając znanego if, dodaj po poprawnym obliczeniu komunikat „Komplet punktów!” dla 100%. Dla innych wyników ten komunikat nie powinien się pojawiać. Sprawdź 20/20, a potem 19/20.

Kolejność pracy

Etap	Nowa umiejętność	Efekt w aplikacji
A	else if, &&, ||, let	Jednoznaczne przypisanie oceny
B	Parametry i return z wartością	Wspólne funkcje obliczeniowe
C	Argument tekstowy prefix	Dwa niezależne kalkulatory
D	Pętla for	Oceny w tabeli HTML

Najpierw dokończ i przetestuj jeden etap, potem przejdź dalej. W tej instrukcji przykłady wyjaśniają narzędzia, a zadania określają wymagane działanie aplikacji.

Wybierz jeden z kilku przypadków

if, else if i else

W łańcuchu if / else if / else warunki są sprawdzane od góry. Wykona się pierwsza pasująca gałąź. Kolejne gałęzie tego samego łańcucha zostaną pominięte. Przykład z zapasem towaru:

```
const stock = 12;
if (stock >= 20) {
  console.log('Duży zapas');
} else if (stock >= 10) {
  console.log('Średni zapas');
} else {
  console.log('Mały zapas');
}
```

Dla stock = 12 pojawi się tylko „Średni zapas”. Drugi warunek nie musi dodawać stock < 20: gdyby zapas wynosił co najmniej 20, wykonałaby się wcześniejsza gałąź.

Operatory logiczne: && oraz ||

Zapis	Jak go odczytać	Kiedy warunek jest prawdziwy
a && b	a oraz b	Gdy oba warunki są prawdziwe
a || b	a lub b	Gdy przynajmniej jeden warunek jest prawdziwy

```
const quantity = 12;
console.log(quantity >= 10 && quantity < 20); // true
console.log(quantity < 0 || quantity > 100);  // false
```

Operator **&&** przydaje się do zapisu przedziału. Operator **||** pozwala połączyć kilka sytuacji błędnych. Każdy porównywany fragment zapisuj w całości; matematyczny zapis 10 <= quantity < 20 nie sprawdza przedziału tak, jak oczekujesz w JS.

Kiedy użyć let?

```
let label = 'Mały zapas';
if (stock >= 10) {
  label = 'Wystarczający zapas';
}
```

let pozwala później przypisać zmiennej inną wartość. Użyj go, gdy najpierw ustawiasz domyślną ocenę, a następnie zmieniasz ją w warunkach. **const** pozostaje dobrym wyborem dla wartości, których ponownie nie przypisujesz.

Kolejność ma znaczenie
Jeśli progi ocen sprawdzasz od najwyższego do najniższego, wynik 96% powinien trafić do pierwszej gałęzi. Kilka oddzielnych if może przypisać ocenę wiele razy - użyj jednego łańcucha.

ETAP A / ZADANIE

Dodaj ocenę do wyniku sprawdzianu

Po poprawnej walidacji i obliczeniu procentów wyznacz ocenę według tabeli. Zaliczenie następuje od 40%, czyli od oceny 2. Ocena 1 oznacza brak zaliczenia, a oceny 2–6 zaliczenie. Kolor komunikatu zależy od zaliczenia.

Procent zdobytych punktów	Ocena
0 ≤ wynik < 40	1
40 ≤ wynik < 55	2
55 ≤ wynik < 70	3
70 ≤ wynik < 85	4
85 ≤ wynik < 95	5
95 ≤ wynik ≤ 100	6

1. Zacznij od najwyższego progu.
W calculateResult() po walidacji i obliczeniu procentów utwórz zmienną grade. Użyj if / else if / else, aby wybrać jedną ocenę. Warunek dla kolejnego progu dopisuj dopiero po poprzednim.

2. Połącz ocenę z dotychczasowym komunikatem.
Dla 35/50 pokaż np. „Wynik: 70% - Sprawdzian zaliczony! Ocena: 4.”. Użyj textContent istniejącego akapitu math-result. Nie twórz dodatkowego elementu w JS.

3. Uporządkuj sprawdzanie pustych pól.
Możesz połączyć dwa warunki pustych pól przez || i pokazać „Uzupełnij oba pola.”. Po błędzie zachowaj return. Jeżeli zostawiasz osobne komunikaty, nadal sprawdzaj oba pola przed obliczeniem.

Sprawdź granice przedziałów

Punkty / maksimum	Oczekiwana ocena	Zaliczenie
39,9 / 100	1	Nie
40 / 100	2	Tak
50 / 100	2	Tak
54,9 / 100 oraz 55 / 100	2 oraz 3	Tak w obu
69,9 / 100 oraz 70 / 100	3 oraz 4	Tak w obu
84,9 / 100 oraz 85 / 100	4 oraz 5	Tak w obu
94,9 / 100 oraz 95 / 100	5 oraz 6	Tak w obu

Użyj separatora dziesiętnego akceptowanego przez przeglądarkę. Wyznaczaj ocenę na podstawie dokładnego procentu, przed ewentualnym zaokrągleniem do wyświetlenia.

Przekaż dane do funkcji i odbierz wynik

Parametr i argument

```
function calculateCost(price, count) {
  return price * count;
}

const total = calculateCost(8, 3);
console.log(total); // 24
```

price i **count** to parametry: nazwy danych odbieranych przez funkcję. Liczby **8** i **3** to argumenty: konkretne wartości przekazane przy wywołaniu. Pierwszy argument trafia do pierwszego parametru, drugi do drugiego.

Parametry są zmiennymi dostępnymi w ciele funkcji. Przy kolejnym wywołaniu, np. calculateCost(5, 2), ta sama funkcja wykona działanie na innych danych. Nie musisz kopiować wzoru.

return z wartością

Znasz już **return;**, który kończy funkcję po błędzie. Zapis **return wyrażenie;** też ją kończy, a dodatkowo przekazuje wynik do miejsca wywołania. Możesz ten wynik zapisać w const lub użyć w kolejnym obliczeniu.

```
const first = calculateCost(8, 3);
const second = calculateCost(5, 2);
console.log(first + second); // 34
```

Konsola i zwracanie wartości
console.log() pokazuje informację w konsoli. return oddaje wartość do dalszego użycia. Jeśli funkcja tylko wypisze liczbę i nie wykona return z wartością, zapisanie jej wyniku do zmiennej da undefined.

Wczesne zakończenie gałęzi

```
function stockLabel(stock) {
  if (stock >= 20) return 'Duży zapas';
  if (stock >= 10) return 'Średni zapas';
  return 'Mały zapas';
}
```

Tutaj osobne if są poprawne: return natychmiast kończy funkcję po trafieniu w odpowiedni warunek. Gałęzie nie nadpisują wyniku. Możesz wykorzystać ten sposób przy porządkowaniu funkcji wyznaczającej ocenę.

Zapisuj zwracaną wartość w tym samym wierszu co return. Zmienna utworzona wewnątrz funkcji jest lokalna; inne funkcje otrzymują potrzebne dane przez argumenty.

ETAP B / ZADANIE

Wydziel obliczenia z obsługi strony

Podziel kod na mniejsze funkcje. Dwie z nich będą wykonywały obliczenia na przekazanych liczbach. calculateResult() nadal obsługuje pola, walidację i komunikaty.

Funkcja	Przyjmuje	Zwraca / wykonuje
calculatePercentage(points, maxPoints)	Dwie poprawne liczby	Procent jako liczbę
gradeFromPercentage(percentage)	Procent od 0 do 100	Ocenę 1-6 jako liczbę
calculateResult(prefix)	Tekst wskazujący zestaw pól	Pobiera dane i pokazuje wynik
clearResult(prefix)	Ten sam tekst	Czyści właściwe pola i komunikat

1. Napisz calculatePercentage(points, maxPoints).
Przenieś do niej sam wzór procentowy. Zwróć obliczoną liczbę przez return. Dane mają już być sprawdzone przez kod wywołujący tę funkcję.

2. Napisz gradeFromPercentage(percentage).
Przenieś do niej wybór oceny. Każda poprawna ścieżka musi zwracać liczbę od 1 do 6. Możesz użyć łańcucha if/else if albo oddzielnych if zakończonych return.

3. Wykorzystaj zwrócone wartości.
W calculateResult() po walidacji wywołaj pierwszą funkcję, a następnie przekaż procent do drugiej. Z wyników zbuduj komunikat. Następnie uzupełnij showExamples(): do istniejącego akapitu examples wpisz oceny dla 40%, 70% i 95%, używając gradeFromPercentage().

Wypróbuj same funkcje w konsoli

```
calculatePercentage(18, 30)  // oczekiwane: 60
gradeFromPercentage(60)      // oczekiwane: 3
gradeFromPercentage(95)      // oczekiwane: 6
```

Dodatkowo: czytelny zapis liczby

Możesz dopisać formatNumber(value), która zwróci tekst z maksymalnie dwoma miejscami po przecinku i polskim zapisem liczby. Przykład użycia metody:

```
const value = 12.345;
const label = value.toLocaleString('pl-PL', {
  maximumFractionDigits: 2
});
console.log(label); // 12,35
```

toLocaleString() zwraca tekst. Używaj go dopiero do prezentacji wyniku. Porównanie z 40% i wybór oceny wykonuj na oryginalnej liczbie. Blok w klamrach to ustawienia formatowania; na tym etapie wystarczy wiedzieć, że maximumFractionDigits ogranicza liczbę miejsc po przecinku.

Dodaj drugi kalkulator

W index.html masz już sekcję „Sprawdzian - matematyka”. Skopiuj tylko tę sekcję i wklej ją pod spodem. Kopię nazwij „Sprawdzian - język polski”. Zachowaj klasy Bootstrap. To jedyna sekcja, którą dodajesz w HTML podczas tej lekcji.

Element	Matematyka	Język polski
Zdobyte punkty	math-points	polish-points
Maksymalne punkty	math-maxPoints	polish-maxPoints
Akapit z wynikiem	math-result	polish-result

W skopiowanej sekcji zmień zarówno **id** pól, jak i odpowiadające im **for** etykiet. Dzięki temu każda etykieta nadal wskazuje właściwe pole, a identyfikatory pozostają unikalne.

Wywołania przycisków w obu sekcjach

```
<!-- Matematyka: przedrostek math- -->
<button class="btn btn-success"
        onclick="calculateResult('math-')">Oblicz wynik</button>
<button class="btn btn-outline-secondary"
        onclick="clearResult('math-')">Wyczyść</button>

<!-- Język polski: przedrostek polish- -->
<button class="btn btn-success"
        onclick="calculateResult('polish-')">
  Oblicz wynik
</button>
<button class="btn btn-outline-secondary"
        onclick="clearResult('polish-')">Wyczyść</button>
```

Co oznacza prefix?

prefix jest parametrem zawierającym początek id. Znak **+** łączy teksty. Każdy przedmiot ma swój prefiks: math- wskazuje matematykę, a polish- język polski. Nazwa pola powstaje z prefiksu i stałej końcówki:

```
'math-' + 'points'    // math-points
'polish-' + 'points'  // polish-points
```

Przyciski w obu sekcjach wywołują te same funkcje. Zmieniają się tylko dane wejściowe wskazujące, z którego zestawu pól funkcja ma skorzystać.

Struktura należy do HTML
Druga sekcja, jej pola, etykiety i przyciski mają być zapisane w index.html. JavaScript odczytuje te elementy i zmienia ich zawartość.

Obsłuż oba zestawy jednym kodem

1. Dodaj parametr prefix.
Zmień definicje na calculateResult(prefix) i clearResult(prefix). W każdej funkcji użyj prefix do zbudowania trzech potrzebnych id: points, maxPoints i result.

```
const pointsInput = document.getElementById(
  prefix + 'points'
);
```

Analogicznie odnajdź pole maksymalnych punktów i akapit wyniku. Wszystkie trzy odwołania muszą używać tego samego prefix. Dla matematyki otrzymasz math-points, math-maxPoints i math-result. Sprawdź też, czy każde wywołanie przycisku przekazuje math- albo polish-.

2. Zachowaj walidację z lekcji 2.
Sprawdź oba puste pola, dodatnie maksimum, nieujemne punkty i przekroczenie maksimum. Puste pola mogą mieć wspólny komunikat „Uzupełnij oba pola.”; pozostałe błędy nazwij jak w lekcji 2. Komunikat pokaż w kliknięte sekcji i zakończ funkcję przez return.

3. Wywołaj wspólne funkcje obliczeniowe.
Po walidacji użyj calculatePercentage() oraz gradeFromPercentage(). Sprawdź zaliczenie od 40%, ustaw textContent i klasy Bootstrap właściwego akapitu. Opcjonalnie sformatuj procent przez formatNumber().

4. Dostosuj czyszczenie.
W clearResult(prefix) ustaw .value obu odnalezionych pól na pusty tekst. Wyczyść komunikat albo przywróć początkowy tekst. Ustaw neutralne klasy: alert alert-light border mt-3 mb-0. W konsoli pokaż „Pola wyczyszczone”.

Zestawy klas pozostają takie jak wcześniej

Sytuacja	Klasy akapitu
Zaliczenie	alert alert-success mt-3 mb-0
Brak zaliczenia lub błąd	alert alert-danger mt-3 mb-0
Po czyszczeniu	alert alert-light border mt-3 mb-0

Test niezależności

W matematyce wpisz 35/50, a w języku polskim 9/20. Najpierw oblicz matematykę, potem polski. Oczekujesz 70%, zaliczenia i oceny 4 w pierwszym zestawie oraz 45%, zaliczenia i oceny 2 w drugim.

Wyczyść wyłącznie polski. Wynik i pola matematyki mają pozostać bez zmian. Następnie wpisz w polskim 30/20 i sprawdź, czy błąd pojawia się tylko tam.

Powtórz działanie pętlą for

Chcesz pokazać oceny dla 0%, 10%, 20% i tak dalej aż do 100%. Zamiast wielokrotnie wpisywać podobną instrukcję, wykonasz ją w pętli ze zmieniającą się liczbą.

```
for (let number = 0; number <= 6; number += 2) {
  console.log(number);
}
// W konsoli: 0, 2, 4, 6
```

Część pętli	Znaczenie
let number = 0	Wartość początkowa; ustawiana raz przed pętlą
number <= 6	Warunek sprawdzany przed każdym powtórzeniem
number += 2	Zmiana licznika po wykonaniu ciała pętli
{ ... }	Instrukcje wykonywane w danym powtórzeniu

number += 2 to skrót zapisu number = number + 2. Licznik jest zadeklarowany przez let, bo jego wartość zmienia się po każdym obiegu. Kiedy warunek stanie się fałszywy, pętla się zakończy.

Liczba może pomóc odnaleźć komórkę HTML

```
const number = 20;
const cellId = 'table-grade-' + number;
console.log(cellId); // table-grade-20
```

Podczas łączenia tekstu z liczbą operator + utworzy tekst. Dzięki temu licznik pętli może wskazać id istniejącej komórki. To ten sam pomysł, którego używasz przy prefix, ale teraz zmienia się końcówka nazwy.

Najpierw przewidź, potem uruchom

Zapisz na kartce wartości licznika dla początku 0, końca 100 i kroku 10. Policz je. Powinno być **11 wartości**, ponieważ uwzględniasz zarówno 0, jak i 100.

Warunek i krok muszą do siebie pasować
Dla tabeli do 100% użyj warunku obejmującego 100. Po każdym powtórzeniu zwiększaj licznik o 10. Jeśli licznik się nie zmienia albo oddala od końca zakresu, pętla może nie zakończyć pracy.

ETAP D / ZADANIE

Zaprogramuj gotową tabelę wyników

Sekcja „Tabela wyników”, przycisk „Uzupełnij tabelę” i wszystkie 11 wierszy istnieją już w otrzymanym HTML. Procenty są wpisane, a komórki ocen czekają na dane. Pracujesz wyłącznie w script.js, w przygotowanej funkcji generateTable().

Mapa istniejących komórek

Procent w HTML	Id komórki do uzupełnienia	Oczekiwana ocena
0%	table-grade-0	1
10%	table-grade-10	1
20%	table-grade-20	1
30%	table-grade-30	1
40%	table-grade-40	2
50%	table-grade-50	2
60%	table-grade-60	3
70%	table-grade-70	4
80%	table-grade-80	4
90%	table-grade-90	5
100%	table-grade-100	6

Uzupełnij generateTable()

1. Ustaw zakres pętli.
Użyj for: od 0 do 100 włącznie, krok 10. Nazwij licznik percentage. W każdym powtórzeniu jego wartość odpowiada jednemu wierszowi tabeli.

2. Znajdź właściwą komórkę.
Połącz tekst table-grade- z wartością licznika. Przekaż uzyskane id do getElementById(). Przykładowo dla 20 otrzymasz table-grade-20.

3. Wpisz ocenę.
Wywołaj gradeFromPercentage(percentage). Zwróconą liczbę wpisz do textContent odnalezionej komórki. Użyj tej samej funkcji oceny, z której korzystają oba kalkulatory.

4. Sprawdź ponowne kliknięcie.
Pierwsze kliknięcie wypełnia wszystkie komórki ocen. Drugie ponownie wpisuje wyniki w te same miejsca. Tabela ma nadal 11 wierszy wyników.

Przycisk jest już połączony z generateTable() przez onclick. Masz też gotowy przycisk „Trzy przykłady”, który wywołuje showExamples(). Jego obsługę uzupełniasz w etapie B.

Sprawdź całą nową funkcjonalność

Co sprawdzasz	Dane / oczekiwany rezultat
Skala ocen	39% → 1; 40% → 2; 55% → 3; 70% → 4; 85% → 5; 95% → 6
Zaliczenie i ocena	39,9% → ocena 1, niezaliczony; 40% → ocena 2, zaliczony
Dwa zestawy	Matematyka 35/50 → 70%, ocena 4; polski 9/20 → 45%, ocena 2
Czyszczenie	Wyczyść polski: matematyka pozostaje bez zmian
Błędy danych	Puste pole, maksimum 0, ujemne punkty i przekroczenie maksimum → błąd we właściwej sekcji
Wiersze tabeli	0, 10, 20, 30 → 1; 40 i 50 → 2; 60 → 3; 70 i 80 → 4; 90 → 5; 100 → 6
Przyciski tabeli	Trzy przykłady → oceny 2, 4 i 6; Uzupełnij tabelę → 11 wierszy wyników

Najczęstsze przyczyny problemów

Zawsze wyświetla się najniższa ocena: sprawdź kolejność progów i to, czy oddzielne if nie nadpisują zmiennej. **Pojawia się undefined:** sprawdź return oraz zapis wyniku wywołania do zmiennej. **Oba przyciski zmieniają ten sam wynik:** sprawdź prefix w trzech odwołaniach do HTML i unikalne id.

Pętla nie uzupełnia 100%: sprawdź warunek końca. **Cannot read properties of null:** porównaj id z HTML z nazwą budowaną w JS, razem z myślnikiem. **Błąd jest nadpisywany wynikiem:** dopisz return po komunikacie walidacji.

Warunki ukończenia

Masz dwa kalkulatory w HTML oraz jedną wspólną funkcję calculateResult(prefix) i jedną clearResult(prefix). Procenty i oceny zwracają calculatePercentage() i gradeFromPercentage(). Funkcja generateTable() używa for do wpisania ocen w stałe komórki HTML. Poprzednia walidacja i kolory nadal działają.

Przygotuj krótką demonstrację: pokaż wywołanie funkcji w konsoli, oblicz różne wyniki w obu zestawach, wyczyść jeden zestaw i uzupełnij tabelę. Wyjaśnij własnymi słowami parametr, argument, return i krok pętli.

Zadanie dodatkowe

W showExamples() zmień przykładowe wyniki na 20%, 60% i 90% i sprawdź oceny 1, 3 i 5. W istniejącym akapicie wyniku dodaj komunikat motywujący zależny od procentu. Oba rozszerzenia wykonaj w JS, korzystając z gotowych elementów HTML.

Dokumentacja: MDN: funkcje · return · if/else · for · toLocaleString.